Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.

1.3. Các nguyên tắc kiểm thử

Trong nhiều năm qua, một số nguyên tắc kiểm thử đã được đề xuất nhằm cung cấp các **hướng dẫn chung** áp dụng cho mọi hoạt động kiểm thử. Giáo trình này mô tả **bảy nguyên tắc** sau đây:

1. **Kiểm thử cho thấy sự hiện diện của khuyết tật, chứ không phải sự vắng mặt của chúng:** Kiểm thử có thể chỉ ra rằng các khuyết tật hiện diện trong đối tượng kiểm thử, nhưng **không thể chứng minh rằng không còn khuyết tật nào**. Kiểm thử giúp **giảm xác suất** các khuyết tật chưa được phát hiện còn sót lại trong đối tượng kiểm thử, nhưng ngay cả khi không tìm thấy khuyết tật, kiểm thử cũng không thể chứng minh tính đúng đắn của đối tượng kiểm thử đó.

2. **Kiểm thử kiệt xuất là không thể:** Việc kiểm thử mọi thứ (tất cả các tổ hợp đầu vào và điều kiện tiền quyết) là **không khả thi**, trừ những trường hợp cực kỳ đơn giản. Thay vì cố gắng kiểm thử kiệt xuất, các **kỹ thuật kiểm thử**, việc **ưu tiên kịch bản kiểm thử** và **kiểm thử dựa trên rủi ro** nên được sử dụng để tập trung các nỗ lực kiểm thử.

3. **Kiểm thử sớm giúp tiết kiệm thời gian và tiền bạc:** Các khuyết tật được loại bỏ sớm trong quy trình sẽ không gây ra các khuyết tật phái sinh trong các sản phẩm công việc tiếp theo. **Chi phí chất lượng** sẽ được giảm thiểu vì sẽ có **ít thất bại hơn** xảy ra trong các giai đoạn sau của vòng đời phát triển phần mềm (SDLC). Để tìm thấy khuyết tật sớm, cả **kiểm thử tĩnh và kiểm thử động** nên được bắt đầu càng sớm càng tốt.

4. **Khuyết tật thường tập trung lại:** Một số lượng nhỏ các thành phần hệ thống thường chứa **hầu hết các khuyết tật** được phát hiện hoặc chịu trách nhiệm cho phần lớn các thất bại vận hành. Hiện tượng này là một minh chứng cho **nguyên lý Pareto**. Các cụm khuyết tật được dự đoán trước, hoặc các cụm khuyết tật thực tế quan sát được trong quá trình kiểm thử hoặc vận hành, là một đầu vào quan trọng cho **kiểm thử dựa trên rủi ro**.

5. **Các bài kiểm tra sẽ bị "lờn" (Tests wear out):** Nếu các bài kiểm tra giống nhau được lặp lại nhiều lần, chúng sẽ dần trở nên **kém hiệu quả** trong việc phát hiện các khuyết tật mới. Để khắc phục hiệu ứng này, các bài kiểm tra và dữ liệu kiểm thử hiện có có thể cần được **sửa đổi**, và các bài kiểm tra

mới cần được viết thêm. Tuy nhiên, trong một số trường hợp, việc lặp lại cùng một bài kiểm tra có thể mang lại kết quả có lợi, chẳng hạn như trong **kiểm thử hồi quy tự động**.

6. **Kiểm thử phụ thuộc vào ngữ cảnh:** Không có một phương pháp tiếp cận duy nhất nào có thể áp dụng phổ quát cho tất cả các hoạt động kiểm thử. Kiểm thử được thực hiện theo **những cách khác nhau trong các ngữ cảnh khác nhau**.

7. **Sự ngụy biện về việc không có lỗi:** Đây là một quan niệm sai lầm khi kỳ vọng rằng việc **xác minh (verification)** phần mềm sẽ đảm bảo sự thành công của một hệ thống. Việc kiểm thử kỹ lưỡng tất cả các yêu cầu đã chỉ định và sửa hết các khuyết tật tìm thấy vẫn có thể tạo ra một hệ thống **không đáp ứng được nhu cầu và kỳ vọng của người dùng**, không giúp đạt được mục tiêu kinh doanh của khách hàng, hoặc kém hơn so với các hệ thống cạnh tranh khác. Bên cạnh việc xác minh, hoạt động **xác nhận (validation)** cũng cần được thực hiện.

1.3. Các nguyên tắc kiểm thử (Testing Principles)

Tổng quan

```
7 NGUYÊN TẮC KIỂM THỬ
Cung cấp hướng dẫn chung cho MỌI hoạt động kiểm thử
```

Bảy Nguyên tắc Kiểm thử

Sơ đồ tổng quan

```
┌──────────────────────────────────────────────────────────────┐
│           7 NGUYÊN TẮC KIỂM THỬ ISTQB                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  1️⃣  Testing shows presence, not absence                    │
│      (Chỉ thấy có lỗi, không chứng minh không có lỗi)        │
│                                                              │
│  2️⃣  Exhaustive testing is impossible                       │
│      (Không thể test hết mọi trường hợp)                     │
│                                                              │
│  3️⃣  Early testing saves time and money                     │
│      (Test sớm = Tiết kiệm)                                  │
│                                                              │
│  4️⃣  Defects cluster together                               │
│      (Lỗi thường tập trung ở vài module)                     │
│                                                              │
│  5️⃣  Tests wear out (Pesticide paradox)                     │
│      (Test cũ sẽ "lờn", cần cập nhật)                        │
│                                                              │
│  6️⃣  Testing is context dependent                           │
│      (Test phụ thuộc ngữ cảnh)                               │
│                                                              │
│  7️⃣  Absence-of-defects fallacy                             │
```

```
│      (Không lỗi ≠ Sản phẩm tốt)                          │
│                                                           │
└──────────────────────────────────────────────────────────┘
```

1️⃣ Testing shows presence, not absence of defects

Nguyên tắc

```
Testing CÓ THỂ:     ✅ Chứng minh CÓ lỗi
Testing KHÔNG THỂ:  ❌ Chứng minh KHÔNG CÓ lỗi
```

Khía cạnh	Chi tiết
Ý nghĩa	Testing chỉ giảm xác suất còn lỗi, không chứng minh hoàn hảo
Tìm 0 bugs	≠ Không còn bugs = Có thể test chưa đủ tốt
Giá trị	Tăng độ tin cậy, nhưng không đảm bảo tuyệt đối

Ví dụ minh họa

```
Tình huống: Test app calculator với 1000 test cases
           ↓
Kết quả: Tất cả PASS ✅
           ↓
Kết luận SAI: "App không có bug"  ❌
Kết luận ĐÚNG: "Với 1000 tests, chưa tìm thấy bug,
                độ tin cậy cao, nhưng vẫn có thể
                có bug ở case chưa test" ✅
```

2️⃣ Exhaustive testing is impossible

Nguyên tắc

```
Exhaustive Testing (Test toàn diện) = KHÔNG KHẢ THI
(Trừ những case cực kỳ đơn giản)
```

Giải pháp thay thế

Thay vì	Nên
❌ Test hết mọi tổ hợp	✅ Sử dụng **kỹ thuật kiểm thử**
❌ Test ngẫu nhiên	✅ **Ưu tiên test cases**
❌ Test đều tất cả	✅ **Kiểm thử dựa trên rủi ro**

Ví dụ

```
Form đăng ký có 5 fields:
├─ Username: 8-20 ký tự
├─ Password: 8-20 ký tự
├─ Email
├─ Phone
└─ Age

Số tổ hợp có thể: HÀNG TỶ TỶ ⚠️

Giải pháp:
├─ Equivalence Partitioning
├─ Boundary Value Analysis
├─ Risk-based Testing
└─ Test ưu tiên: Security, critical paths
```

Tham khảo

- **Kỹ thuật kiểm thử**: Chương 4
- **Ưu tiên test case**: Section 5.1.5
- **Kiểm thử dựa trên rủi ro**: Section 5.2

3️⃣ Early testing saves time and money

Nguyên tắc

```
Test càng sớm → Chi phí càng thấp

Requirements: $1
Design: $5
Development: $10
Testing: $15
Production: $100
```

Lợi ích

Lợi ích	Giải thích
💰 **Giảm chi phí**	Fix lỗi sớm rẻ hơn 100x so với production
⏰ **Tiết kiệm thời gian**	Không phải làm lại nhiều
🛡️ **Ngăn lỗi phái sinh**	Lỗi sớm không gây thêm lỗi sau
📉 **Giảm failures**	Ít thất bại trong SDLC sau

Cách áp dụng

```
Bắt đầu càng sớm càng tốt:
├─ 📄 Static Testing: Review requirements, design
└─ 💻 Dynamic Testing: Unit test ngay khi code
```

4️⃣ Defects cluster together

Nguyên tắc (Nguyên lý Pareto)

```
80/20 Rule:
80% bugs đến từ 20% modules
```

Hiện tượng	Giải thích
Defect Clustering	Bugs tập trung ở một số modules
Nguyên nhân	Code phức tạp, dev mới, logic khó
Ứng dụng	Đầu vào cho **Risk-based Testing**

Ví dụ thực tế

```
Dự án có 50 modules:
├─ Module Payment: 120 bugs (40%)
├─ Module Login: 80 bugs (27%)
├─ Module Search: 60 bugs (20%)
└─ 47 modules còn lại: 40 bugs (13%)

→ Tập trung test kỹ vào Payment, Login, Search!
```

Hành động

```
Phát hiện Defect Clusters:
├─ Dự đoán trước (Based on risk analysis)
├─ Quan sát thực tế (During testing)
└─ Phân bổ effort: Focus vào high-risk areas
```

5️⃣ Tests wear out (Pesticide Paradox)

Nguyên tắc

```
Test cases giống nhau lặp đi lặp lại
        ↓
Trở nên KÉM HIỆU QUẢ tìm bugs mới
        ↓
Cần CẬP NHẬT và BỔ SUNG tests
```

Vấn đề	Giải pháp
Test cũ không tìm bugs mới	✅ Sửa đổi test data
	✅ Viết test cases mới
	✅ Thay đổi test approach

Ngoại lệ: Khi lặp lại có lợi

```
Regression Testing tự động:
└─ Lặp lại SAME tests → Đảm bảo không bị regression
   → Đây là MONG MUỐN ✅
```

Ví dụ

```
Sprint 1: 50 test cases → Tìm 30 bugs
Sprint 2: SAME 50 tests → Tìm 5 bugs
Sprint 3: SAME 50 tests → Tìm 1 bug

⚠️ Tests đã "lờn" - cần làm mới!

Giải pháp:
├─ Review và update test data
├─ Thêm edge cases
├─ Test những kịch bản mới
└─ Exploratory testing
```

6️⃣ Testing is context dependent

Nguyên tắc

```
KHÔNG CÓ phương pháp ONE-SIZE-FITS-ALL
Testing khác nhau tùy ngữ cảnh
```

Các yếu tố ngữ cảnh

```
Testing approach phụ thuộc vào:
├─ 📱 Loại ứng dụng (Mobile/Web/Embedded)
├─ 🏭 Ngành (Banking/Gaming/Healthcare)
├─ 🔄 SDLC (Agile/Waterfall/DevOps)
├─ ⚠️ Rủi ro (Safety-critical/Commercial)
├─ 👥 Team size
├─ ⏰ Timeline
└─ 💰 Budget
```

Ví dụ so sánh

Ngữ cảnh	Testing approach
Banking app	• Security testing +++ • Compliance testing • Slow release cycle • Extensive documentation
Gaming app	• Performance testing +++ • UX testing

	• Fast iteration • Less documentation
Medical device	• Safety testing +++ • Regulatory compliance • Formal verification • Traceability required
E-commerce	• Load testing • Payment security • A/B testing • Fast deployment

Thực hành

```
E-commerce website:
├─ Focus: Performance, UX, Security
├─ Methods: A/B testing, Load testing
└─ Speed: Fast release

Medical device software:
├─ Focus: Safety, Reliability
├─ Methods: Formal verification, Extensive testing
└─ Speed: Slow, thorough validation
```

7️⃣ Absence-of-defects fallacy

Nguyên tắc

```
Không có lỗi ≠ Sản phẩm thành công

Verification (làm đúng) + Validation (làm đúng cái cần)
= Thành công
```

Ngụy biện

Ngụy biện	Thực tế
Test kỹ + Fix hết bugs = Thành công	❌ Có thể làm ĐÚNG sản phẩm SAI
100% test coverage = Tốt	❌ Nếu test sai requirements
Pass hết test cases = Release	❌ Nếu không đáp ứng nhu cầu user

Ví dụ kinh điển: Bộ Vest vs Đồ Bơi 🤵🏊

```
┌──────────────────────────────────────────────┐
│          MINH HỌA NGUYÊN TẮC 7               │
├──────────────────────────────────────────────┤
│                                              │
```

```
│  KHÁCH HÀNG CẦN: Bộ đồ bơi 🏊                        │
│                                                       │
│  NHƯNG requirements ghi: "Bộ vest"                   │
│                                                       │
│  THỢ MAY:                                            │
│  ├─ May bộ vest HOÀN HẢO 👔                          │
│  ├─ Không một sợi chỉ thừa                           │
│  ├─ Đúng từng milimet                                │
│  └─ ✅ VERIFICATION: PASS 100%                       │
│                                                       │
│  NHƯNG:                                              │
│  └─ ❌ VALIDATION: FAIL                              │
│      (Không đáp ứng nhu cầu thực tế)                 │
│                                                       │
│  KẾT QUẢ:                                            │
│  └─ Sản phẩm "không có lỗi" nhưng THẤT BẠI           │
│                                                       │
└───────────────────────────────────────────────────────┘
```

Ví dụ thực tế

Tình huống	Verification ✅	Validation ❌	Kết quả
App đặt hàng	100% đúng requirements	UI xấu, chậm, khó dùng	User chuyển sang đối thủ
CRM software	Đầy đủ 50 tính năng	Thiếu tính năng export báo cáo (cần thiết)	Khách hàng không hài lòng
Website	Không bugs, đẹp	Load chậm hơn đối thủ 3 giây	Mất traffic

Giải pháp

```
Không chỉ VERIFICATION
        ↓
Phải có cả VALIDATION
        ↓
Đảm bảo sản phẩm:
├─ ✅ Đúng yêu cầu (Verification)
├─ ✅ Đáp ứng nhu cầu thực (Validation)
├─ ✅ Đạt mục tiêu kinh doanh
└─ ✅ Cạnh tranh tốt với đối thủ
```

Bảng Tóm tắt 7 Nguyên tắc

#	Nguyên tắc	Ý nghĩa	Hành động
1️⃣	**Shows presence, not absence**	Chỉ tìm lỗi, không chứng minh hoàn hảo	Chấp nhận rủi ro còn sót lỗi

2️⃣	**Exhaustive testing impossible**	Không thể test hết	Dùng kỹ thuật, ưu tiên, risk-based
3️⃣	**Early testing saves**	Test sớm = Tiết kiệm	Static + Dynamic testing sớm
4️⃣	**Defects cluster**	80% bugs từ 20% modules	Focus vào high-risk areas
5️⃣	**Tests wear out**	Test cũ kém hiệu quả	Cập nhật, bổ sung tests
6️⃣	**Context dependent**	Mỗi ngữ cảnh khác nhau	Điều chỉnh approach theo context
7️⃣	**Absence-of-defects fallacy**	Không lỗi ≠ Tốt	Verification + Validation

---

Mẹo Ghi nhớ 7 Nguyên tắc

```
S.E.E.D.T.C.A

S - Shows presence (not absence)
E - Exhaustive testing (impossible)
E - Early testing (saves money)
D - Defects cluster
T - Tests wear out
C - Context dependent
A - Absence-of-defects (fallacy)
```

Ứng dụng Thực tế

```
Khi làm dự án, áp dụng 7 nguyên tắc:

PLANNING PHASE:
├─ Nguyên tắc 2: Không test hết → Ưu tiên risk-based
├─ Nguyên tắc 3: Lên kế hoạch test sớm
├─ Nguyên tắc 4: Phân tích defect clusters
└─ Nguyên tắc 6: Điều chỉnh theo context

EXECUTION PHASE:
├─ Nguyên tắc 1: Tìm bugs, nhưng biết không thể tìm hết
├─ Nguyên tắc 5: Cập nhật tests thường xuyên
└─ Nguyên tắc 7: Đảm bảo cả verification & validation

REVIEW & RELEASE:
├─ Nguyên tắc 1: Chấp nhận rủi ro hợp lý
└─ Nguyên tắc 7: Kiểm tra user needs, business goals
```

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
    - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
    - Follow Fanpage: https://www.facebook.com/hoctest/
    - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
    - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
    - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.